CS 61A
Past Exam Practice — Linked Lists and Trees

Your Name: _____

Your Student ID: _____

Room you are taking the exam in: _____

Name and SID of the person to your **left**: _____

Name and SID of the person to your **right**: _____

You have 110 minutes. There are 6 questions of varying credit. (47 points total)

Question	1	2	3	4	5	6	Total
Points	5	4	6	8	15	9	47

For questions with **circular bubbles**, you may select only one choice.

- ○ Unselected option (Completely unfilled)
- ◎ Don't do this (it will be graded as incorrect)
- ● Only one selected option (completely filled)

For questions with **square checkboxes**, you may select one or more choices.

- ■ You can select
- ■ multiple squares
- ☑ (Don't do this)

Make sure to **write your student ID at the top of each page** in the provided blank.

Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation. For coding questions with blanks, you may write at most one statement per blank and you may not use more blanks than provided.

As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others. I will follow the rules of this exam. I will not use any other unauthorized materials or devices during this exam. I will not communicate with anyone other than the proctors during this exam. I will not share any information about the exam with anyone until after the end of the exam period.

Acknowledge that you have read and agree to the honor code above and **sign your name below**:

_____

This content is protected and may not be shared, uploaded, or distributed.

Reference: New `Link` and `Tree` Types

Use these definitions for every question in this worksheet.

```
from __future__ import annotations
from dataclasses import dataclass, field

type LinkedList[T] = Link[T] | tuple[()]

@dataclass
class Link[T]:
    first: T
    rest: LinkedList[T] = ()

    def __str__(self):
        return format_link(self)

def format_link(s: Link):
    string = '(' + str(s.first)
    remaining = s.rest
    while isinstance(remaining, Link):
        string += ' ' + str(remaining.first)
        remaining = remaining.rest
    assert remaining == (), f'{s!r} is not a LinkedList'
    return string + ')'

@dataclass
class Tree[T]:
    label: T
    branches: list[Tree[T]] = field(default_factory=list)

    def __str__(self):
        return format_tree(self)

def format_tree(t: Tree, indent='') -> str:
    assert isinstance(t, Tree), f'{t!r} is not a Tree'
    assert isinstance(t.branches, list), f'branches of {t!r} is not a list'
    string = indent + str(t.label)
    for b in t.branches:
        string += '\n' + format_tree(b, indent + '  ')
    return string

def is_leaf(t: Tree) -> bool:
    return not t.branches
```


This content is protected and may not be shared, uploaded, or distributed.

Q1 *Untangle* **(5 points)**

Original source: Summer 2023 - Final - Question 3(b) | **Level:** Regular

Implement `untangle`, a function that takes in a linked list `s` with strictly positive values. The function **mutates** the linked list to contain only `1`s, where the length of the resulting linked list is equal to the sum of the values in the original list. `untangle` potentially modifies the values in `s` and adds new `Link`s to `s`.

For example, calling `untangle` on `(3 1 2)` will mutate the linked list to become `(1 1 1 1 1 1)`. See the doctests for more examples.

```
def untangle(s):
    """Mutate s to contain only 1s and preserve the sum as its length.

    >>> a = Link(5)
    >>> untangle(a)
    >>> print(a)
    (1 1 1 1 1)
    >>> b = Link(3, Link(2, Link(4)))
    >>> untangle(b)
    >>> print(b)
    (1 1 1 1 1 1 1 1 1)
    >>> c = Link(1)
    >>> untangle(c)
    >>> print(c)
    (1)
    """
    if _________:
       # (a)
        return
    next = _________
           # (b)
    while _________:
          # (c)
        s.rest = _________
                 # (d)
        s.first = _________
                  # (e)

    _________
    # (f)
```

Q1.1 (1 point) Fill in blank (a).

Q1.2 (1 point) Fill in blank (b).

Q1.3 (0.5 points) Fill in blank (c).

- ◯ `isinstance(s, Link)`
- ◯ `isinstance(s.rest, Link)`
- ◯ `isinstance(s, Link) and isinstance(s.rest, Link)`
- ◯ `isinstance(s, Link) or isinstance(s.rest, Link)`
- ◯ `s.first > 0`
- ◯ `s.first < 0`
- ◯ `s.first > 1`
- ◯ `s.first < 1`

Q1.4 (1 point) Fill in blank (d).

This content is protected and may not be shared, uploaded, or distributed.

(Question 1 continued…)

- ◯ `1`
- ◯ `s`
- ◯ `Link(1)`
- ◯ `Link(1, s)`
- ◯ `Link(1, s.rest)`
- ◯ `Link(s, Link(1))`
- ◯ `Link(s.first, Link(1))`

Q1.5 (0.5 points) Fill in blank (e).

- ◯ `s`
- ◯ `s.rest`
- ◯ `s.rest.first`
- ◯ `s.first - 1`
- ◯ `s.first + 1`
- ◯ `-1 * s.first`

Q1.6 (1 point) Fill in blank (f).

This content is protected and may not be shared, uploaded, or distributed.

Q2 *Their Mascot is a Tree?!?* **(4 points)**

Original source: Fall 2019 - Midterm 2 - Question 8 | **Level:** Regular

Definitions. The depth of the root node of a tree is 0, and the depth of a child is 1 more than the depth of its parent. A linked list is either a `Link` instance or `()`.

Implement `layer`, which takes a `Tree` instance `t` and a non-negative integer `d`. It returns a linked list containing the labels of all nodes in `t` that have depth `d`. Labels in the result should appear in branch order.

You may **not** use `lambda`, `if`, `and`, or `or` in your solution other than what already appears in the template.

Note: The built-in `reversed` function returns a reverse iterator over the values of a sequence. For example, `list(reversed([1, 2, 3]))` evaluates to `[3, 2, 1]`.

```
def layer(t, d):
    """Return a linked list containing all labels of Tree t at depth d.

    >>> a_tree = Tree(1, [Tree('b', [Tree('mas')]),
    ...                   Tree('a', [Tree('co')]),
    ...                   Tree('d', [Tree('t', [Tree('!')])])])
    >>> print(layer(a_tree, 0))
    (1)
    >>> print(layer(a_tree, 1))
    (b a d)
    >>> print(layer(a_tree, 2))
    (mas co t)
    >>> print(layer(a_tree, 3))
    (!)
    """
    return helper(t, d, _________)
                        # (a)

def helper(t, d, s):
    if d == 0:
        return _________
               # (b)
    else:
        for b in reversed(t.branches):
            s = _________
                # (c)
        return s
```

Q2.1 (1 point) Fill in blank (a).

Q2.2 (1 point) Fill in blank (b).

Q2.3 (2 points) Fill in blank (c).

This content is protected and may not be shared, uploaded, or distributed.

Q3 *Merry-Go-Round* **(6 points)**

Original source: Spring 2024 - Final - Question 4(a) | **Level:** Regular

Implement `cycle`, which takes a non-empty finite linked list `s` (a `Link` instance). It returns a linked list in which the first `Link` of `s` has been moved to become the last `Link` of the result. If `s` has only one element, then it is not moved. A linked list is finite if it contains a `Link` whose `rest` attribute is `()` rather than having a cyclical structure.

```
def cycle(s):
    """Move the first Link of a non-empty finite linked list to be the last.

    >>> a = Link(3, Link(5, Link(7, Link(9))))
    >>> print(cycle(a))
    (5 7 9 3)
    >>> print(a)
    (3)
    """
    assert isinstance(s, Link)
    if not isinstance(s.rest, Link):
        return s
    last = s
    while isinstance(last.rest, Link):
        last = last.rest
    s.rest, last.rest, result = _______ , _______ , _______
                                  # (a)    # (b)    # (c)
    return result
```

Q3.1 (2 points) Fill in blank (a).

- ◯ `s`
- ◯ `s.rest`
- ◯ `last`
- ◯ `last.rest`
- ◯ `()`

Q3.2 (2 points) Fill in blank (b).

- ◯ `s`
- ◯ `s.rest`
- ◯ `last`
- ◯ `last.rest`
- ◯ `()`

Q3.3 (2 points) Fill in blank (c).

- ◯ `s`
- ◯ `s.rest`
- ◯ `last`
- ◯ `last.rest`
- ◯ `()`

This content is protected and may not be shared, uploaded, or distributed.

Q4 *Promote Link* **(8 points)**

Original source: Fall 2024 - Final - Question 4(c) | **Level:** Challenge

Implement `promote_link`, which takes a `Link` instance `s` (a non-empty linked list) and two non-negative integers `i` and `j` with `i < j` and `j` less than the length of `s`. It mutates `s` by promoting to `i` from `j` and then returns `s`.

```
def promote_link(s, i, j):
    """Mutate linked list s so that the item at index j is at index i.

    >>> a = Link(3, Link(6, Link(9, Link(12, Link(15, Link(18))))))
    >>> print(promote_link(a, 2, 4))
    (3 6 15 9 12 18)
    >>> print(promote_link(a, 0, 4))
    (12 3 6 15 9 18)
    >>> promote_link(a, 1, 3) is a
    True
    """
    assert i >= 0 and i < j
    if i > 0:
        _______
        # (g)
    else:
        insert, tail = s.first, s.rest
        while j > 0:
            _______  # Hint: use multiple assignment
            # (h)
            tail, j = tail.rest, j - 1
        _______ = insert
        # (i)
    return _______
           # (j)
```

Q4.1 (2 points) Fill in blank (g).

- ◯ `promote_link(s.rest, i - 1, j)`
- ◯ `return promote_link(s.rest, i - 1, j)`
- ◯ `promote_link(s.rest, i, j - 1)`
- ◯ `return promote_link(s.rest, i, j - 1)`
- ◯ `promote_link(s.rest, i - 1, j - 1)`
- ◯ `return promote_link(s.rest, i - 1, j - 1)`

Q4.2 (2 points) Fill in blank (h). **Hint:** Use multiple assignment: ___, ___ = ___, ___.

Q4.3 (1 point) Fill in blank (i).

- ◯ `s.first`
- ◯ `tail.first`
- ◯ `s.rest.first`
- ◯ `tail.rest.first`

Q4.4 (1 point) Fill in blank (j).

- ◯ `s`
- ◯ `Link(insert, tail.rest)`
- ◯ `tail`
- ◯ `Link(s.first, tail.rest)`

Q4.5 (2 points) What is displayed by the call to `print` in this code?

This content is protected and may not be shared, uploaded, or distributed.

(Question 4 continued…)

```
odds = Link(3, Link(5, Link(7, Link(9))))
for i in range(3):
    promote_link(odds, 0, 3)
print(odds)
```

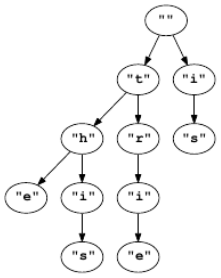
- ◯ (3 5 7 9)
- ◯ (3 9 7 5)
- ◯ (5 7 9 3)
- ◯ (5 9 7 3)
- ◯ (9 7 5 3)
- ◯ (9 3 5 7)

This content is protected and may not be shared, uploaded, or distributed.

Q5 *Trie This* **(15 points)**

Original source: Spring 2019 - Midterm 2 - Question 6 | **Level:** Challenge

A **Trie** is a `Tree` where every node contains a single letter except for the root, which is always the empty string. Every path from the root to a leaf forms a word. You may assume no words are substrings of other words in the trie, such as `"hi"` and `"him"`. The figure below is a trie generated by storing the words `["this", "is", "the", "trie"]`.



Q5.1 (7 points) Implement `add_word`, which takes a Trie and a word and adds the word to the trie.

```
def make_trie(words):
    """Make a tree where every node is a letter of a word.
    All words end as a leaf of the tree.
    """
    trie = Tree('')
    for word in words:
        add_word(trie, word)
    return trie

def add_word(trie, word):
    if _________:
       # (a)
        return
    branch = __________
             # (b)
    for _________:
        # (c)
        if _________:
           # (d)
            branch = _________
                     # (e)
    if _________:
       # (f)
        branch = _________
                 # (g)

        _________
        # (h)

    _________
    # (i)
```

(a)

(b)

(c)

(d)

(e)

(f)

This content is protected and may not be shared, uploaded, or distributed.

(Question 5 continued…)

(g)

(h)

(i)

Q5.2 (8 points) Implement `get_words`, which takes a Trie and returns a list of all the words the Trie is storing.

```
def get_words(trie):
    """
    >>> get_words(make_trie(['this', 'is', 'the', 'trie']))
    ['this', 'the', 'trie', 'is']
    """
    if _________:
       # (j)
        return _________
               # (k)
    return sum(_________, [])
               # (l)
```

(j)

(k)

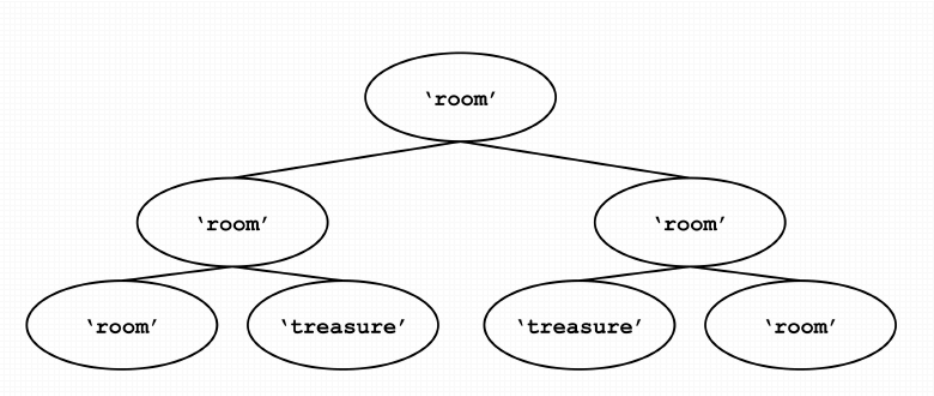
(l)

This content is protected and may not be shared, uploaded, or distributed.

Q6 *Where's the Treasure?* **(9 points)**

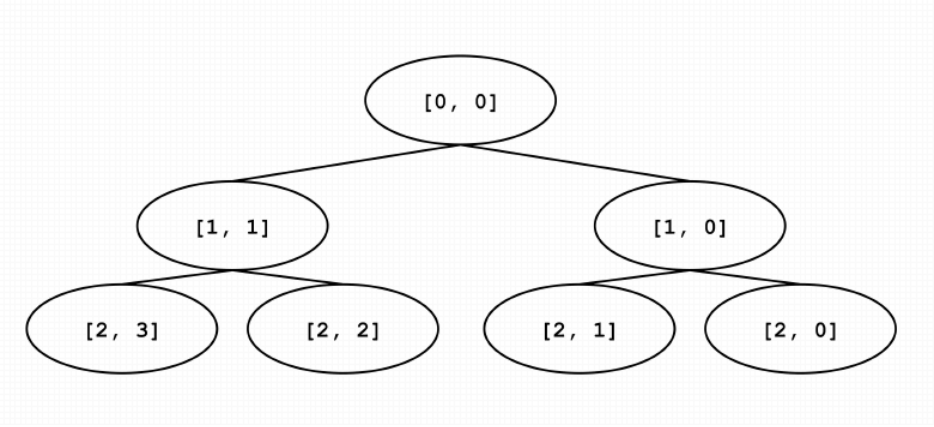
Original source: Spring 2026 - Final - Question 4 | **Level:** Challenge

The function `treasure_locations` accepts a single parameter `t`, an instance of the `Tree` class where all non-leaf nodes have two branches, the label of each node is either `'room'` or `'treasure'`, and only leaf nodes can have the `'treasure'` label. Here's a drawing of an example input tree:



The function returns a list of locations of treasures in `t`, where each location is a two-element list with the "depth" as the first element and the "right" as the second element. The "depth" of the root node is 0, and it increases by one at each level of the tree down from the root node. The right-most node of each level of the tree has a "right" of 0, and it increases from there, so it represents the number of nodes on that level that are to the right of the given node.

Here's the same tree where each node is labeled with its location:



When `treasure_locations` is called on the example tree, it returns `[[2, 2], [2, 1]]`, since the two treasures are located at `[2, 2]` and `[2, 1]`.

Q6.1 (9 points) Complete the `treasure_locations` function below according to the doctests and description.

Note: `sum([[2, 3]], [1]) == [1, 2, 3]`; the sum starts at `[1]`, then adds `[2, 3]`.

This content is protected and may not be shared, uploaded, or distributed.

```
def treasure_locations(t):
    """Return the [depth, right] location of each treasure in t.

    >>> t1 = Tree('room', [Tree('treasure'), Tree('room')])
    >>> print(t1)
    room
      treasure
      room
    >>> treasure_locations(t1)
    [[1, 1]]
    >>> t2 = Tree('room', [Tree('room', [Tree('room'), Tree('treasure')]),
    ...                    Tree('room', [Tree('treasure'), Tree('room')])])
    >>> treasure_locations(t2)
    [[2, 2], [2, 1]]
    >>> no_treasures = Tree('room', [Tree('room'),
    ...                              Tree('room', [Tree('room'), Tree('room')])])
    >>> treasure_locations(no_treasures)
    []
    """
    def helper(t, depth, right):
        if t.label == 'treasure':
            return _________
                   # (a)
        locations = [helper(
            _________,
            # (b)
            _________,
            # (c)
            _________ + len(t.branches) - 1 - i
            # (d)
        ) for i in _________]
                   # (e)
        return sum(locations, [])

    _________
    # (f)
```

(a)

(b)

(c)

(d)

(e)

(f)

This content is protected and may not be shared, uploaded, or distributed.